

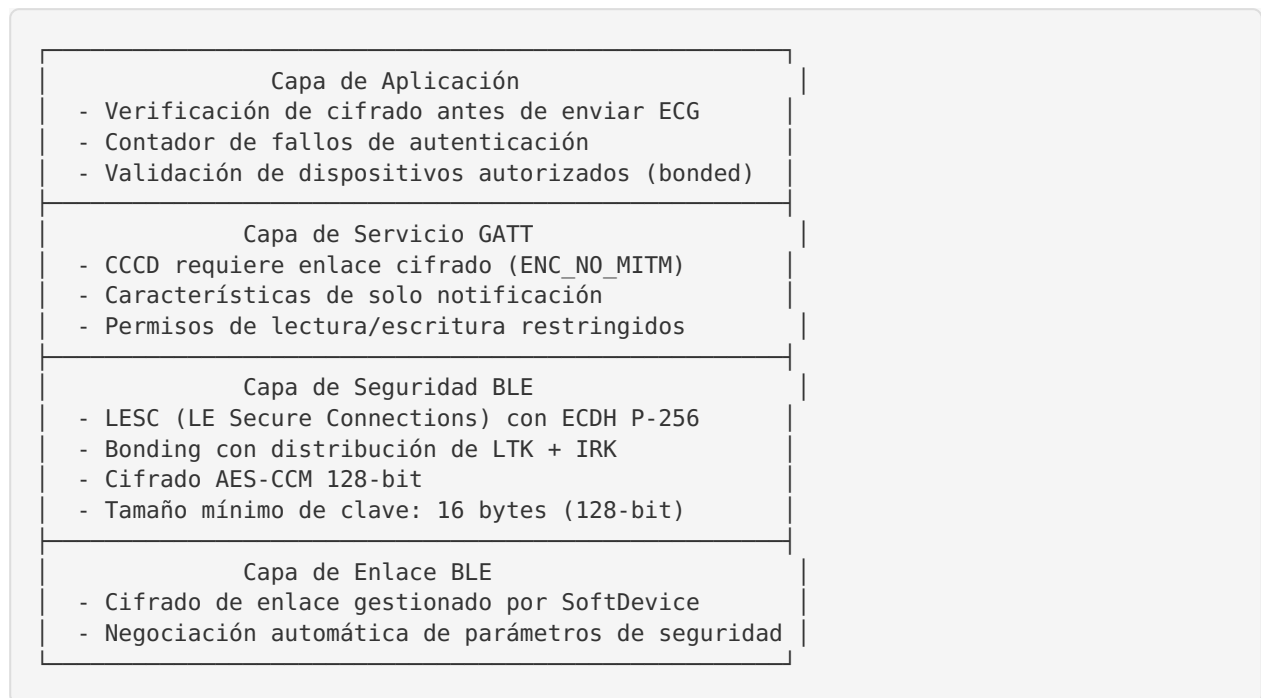
Seguridad BLE - Horizon Medical IoT Holter v2.0

Resumen

Este documento describe la implementación de seguridad avanzada en la comunicación BLE del firmware del Holter IoT de Horizon Medical. El objetivo es proteger la transmisión de datos médicos sensibles (señales ECG) cumpliendo con estándares de privacidad como HIPAA y regulaciones de dispositivos médicos.

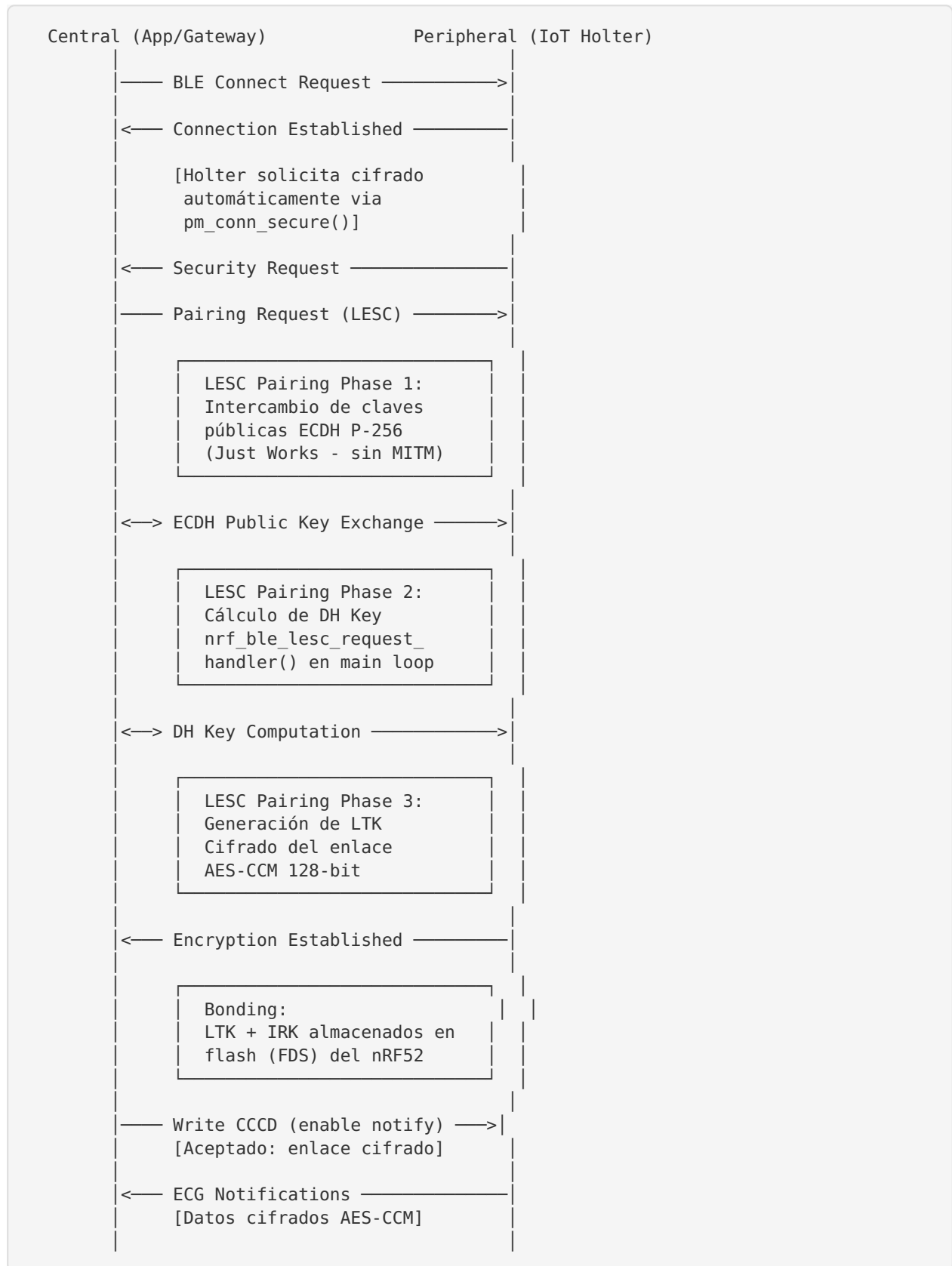
Arquitectura de Seguridad

Capas de Protección

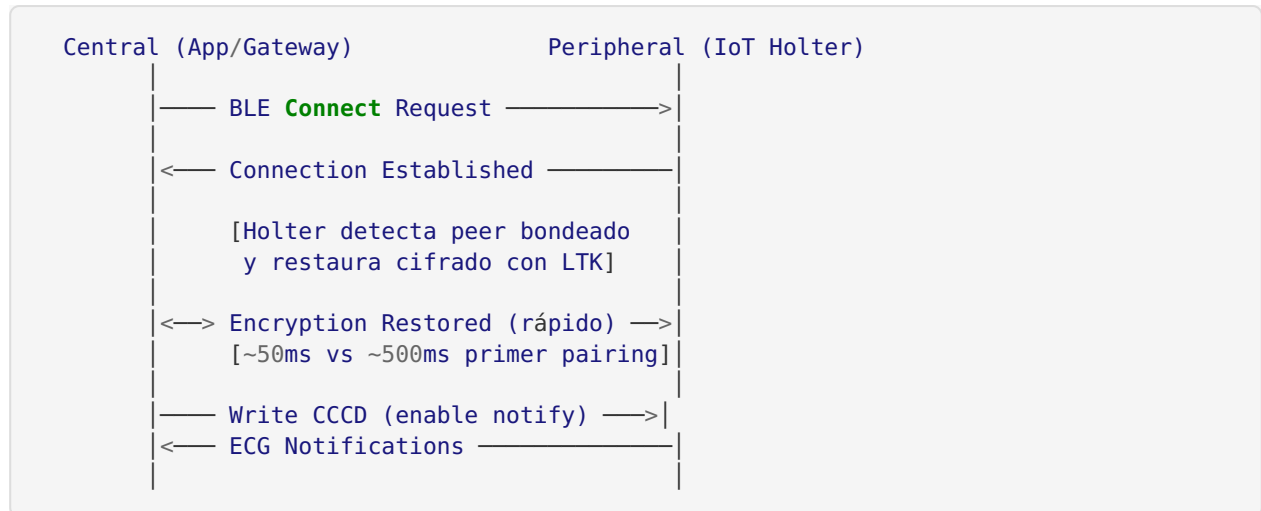


Flujo de Autenticación BLE

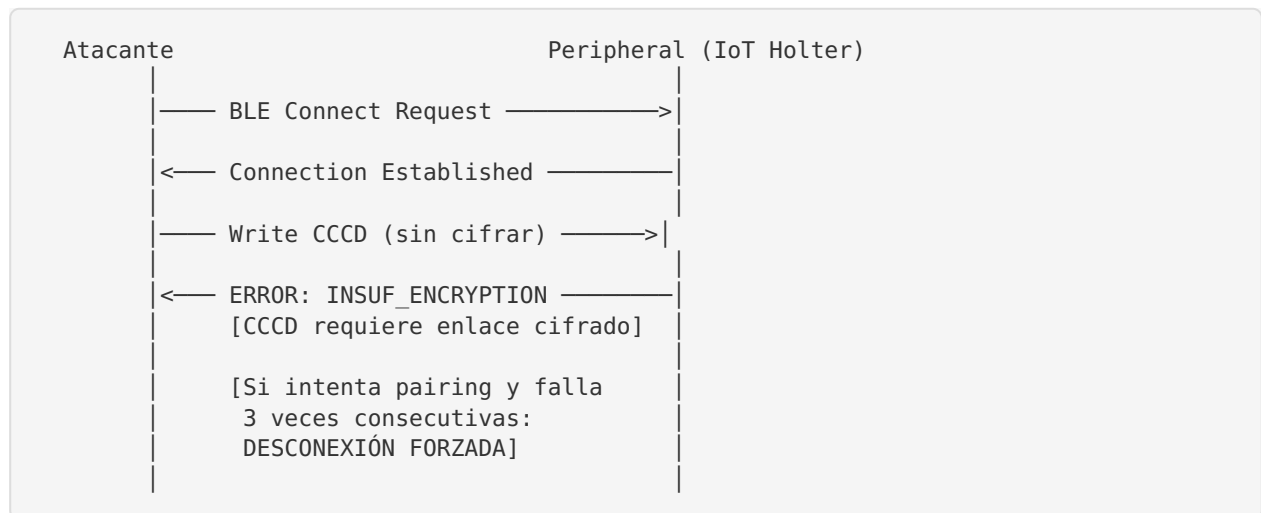
Conexión Nueva (Primer Pairing)



Reconexión (Peer Bondeado)



Intento de Acceso No Autorizado



Cambios Implementados

1. Parámetros de Seguridad (HZM_BLE.cpp)

Parámetro	Antes	Después	Justificación
SEC_PARAM_LESC	0 (deshabilitado)	1 (habilitado)	ECDH P-256 para intercambio seguro de claves
SEC_PARAM_BOND	1	1 (sin cambio)	Persistencia de claves entre sesiones
SEC_PARAM_MITM	0	0 (sin cambio)	Just Works (dispositivo sin display/teclado)
SEC_PARAM_MIN_KEY_SIZE	7	16	Forzar cifrado de 128-bit
PM_LESC_ENABLED	0	1	Habilitar LESC en Peer Manager (sdk_config.h)

2. Permisos de Características ECG

Permiso	Antes	Después
CCCD Read	SET_OPEN	SET_ENC_NO_MITM
CCCD Write	SET_OPEN	SET_ENC_NO_MITM
Char Value Read	SET_NO_ACCESS	SET_NO_ACCESS (sin cambio)
Char Value Write	SET_NO_ACCESS	SET_NO_ACCESS (sin cambio)

3. Nuevo Módulo: HZM_BLE_Security

Archivos:

- HZM_BLE_Security.h - Definiciones, estados, tipos de eventos
- HZM_BLE_Security.cpp - Implementación completa

Funcionalidades:

- Inicialización de LESC (nrf_ble_leesc_init())
- Manejo de eventos del Peer Manager (pairing, bonding, cifrado)
- Manejo de eventos GAP de seguridad (SEC_PARAMS_REQUEST, AUTH_STATUS, etc.)
- Verificación de cifrado de enlace (is_link_encrypted())
- Solicitud de cifrado (request_encryption())
- Validación de dispositivos autorizados (peers bondeados)

- Contador de fallos de autenticación con desconexión forzada
- Callbacks de eventos de seguridad para la aplicación

4. Protección en Transmisión de Datos

`send_data_over_BLE()` ahora verifica el estado de cifrado antes de enviar:

```
if (!HZM_BLE_Security::is_link_encrypted(m_conn_handle))
{
    // Silenciosamente descarta datos - enlace no cifrado
    HZM_AFE::data_read = false;
    return;
}
```

5. Main Loop: LESC Request Handler

```
// Procesar solicitudes de cómputo DH key del SoftDevice
nrf_ble_lesc_request_handler();
```

Crítico: Esta llamada es obligatoria cuando `PM_LESC_ENABLED=1`. El SoftDevice delega el cómputo ECDH al CPU de la aplicación y `nrf_ble_lesc_request_handler()` debe llamarse en el main loop para completar el intercambio de claves durante el pairing.

Eventos de Seguridad

Eventos del Peer Manager

Evento	Acción
<code>PM_EVT_CONN_SEC_START</code>	Log + notifica app que pairing inició
<code>PM_EVT_CONN_SEC_SUCCEEDED</code>	Actualiza estado a ENCRYPTED, reset contador fallos
<code>PM_EVT_CONN_SEC_FAILED</code>	Incrementa contador, desconecta si ≥ 3 fallos
<code>PM_EVT_CONN_SEC_CONFIG_REQ</code>	Permite re-pairing si < 3 fallos
<code>PM_EVT_BONDED_PEER_CONNECTED</code>	Log de reconexión de peer bondeado
<code>PM_EVT_PEER_DATA_UPDATE_SUCCEEDED</code>	Log de bond almacenado en flash
<code>PM_EVT_STORAGE_FULL</code>	Ejecuta garbage collection de FDS

Eventos GAP de Seguridad

Evento	Acción
<code>BLE_GAP_EVT_SEC_PARAMS_REQUEST</code>	Log para audit trail
<code>BLE_GAP_EVT_AUTH_STATUS</code>	Log resultado de autenticación (bonded, LESC, MITM)
<code>BLE_GAP_EVT_CONN_SEC_UPDATE</code>	Verifica nivel de seguridad, autoriza/deniega ECG
<code>BLE_GAP_EVT_AUTH_KEY_REQUEST</code>	Log (no esperado en Just Works)
<code>BLE_GAP_EVT_PHY_UPDATE_REQUEST</code>	Acepta actualización de PHY

Defensa en Profundidad

La seguridad BLE se implementa en **tres capas independientes**:

1. **Capa GATT (permisos):** El SoftDevice rechaza writes al CCCD si el enlace no está cifrado. Esto es la primera línea de defensa y funciona a nivel de protocolo BLE.
2. **Capa de aplicación (send_data_over_BLE):** Verificación explícita del estado de cifrado antes de cada transmisión de datos ECG. Si un bug permitiera suscribirse sin cifrado, esta capa lo bloquea.
3. **Capa de sesión (auth failure counter):** Después de 3 fallos consecutivos de autenticación, el dispositivo desconecta al peer. Protege contra ataques de fuerza bruta.

Consideraciones para Desarrollo Futuro

Mejoras Posibles

1. **MITM Protection:** Si se añade un display al dispositivo, cambiar `SEC_PARAM_IO_CAPABILITIES` a `BLE_GAP_IO_CAPS_DISPLAY_ONLY` y `SEC_PARAM_MITM` a `1` para habilitar Numeric Comparison.
2. **Whitelist de Advertising:** Usar `ble_advertising_whitelist_reply()` para limitar advertising solo a peers bondeados después del primer pairing.
3. **OOB Pairing:** Para entornos de alta seguridad, implementar pairing Out-of-Band usando NFC (el nRF52832 tiene NFC-A integrado).
4. **Rotación de Claves:** Implementar re-pairing periódico para rotar las claves de cifrado.
5. **Secure DFU:** Implementar Device Firmware Update seguro con firma criptográfica para actualizaciones OTA.

Dependencias

Módulo	Descripción
nrf_ble_lesc	Gestión de claves ECDH para LESC
peer_manager	Almacenamiento de bonds en flash
fds	Flash Data Storage para persistencia
nrf_crypto	Backend criptográfico (CC310 o software)
ble_conn_state	Seguimiento de estado de conexiones

Impacto en Recursos

Recurso	Impacto Estimado
Flash	+4-8 KB (código LESC + crypto)
RAM	+2 KB (buffers ECDH P-256)
CPU	Pico durante pairing (~100ms para ECDH)
Latencia conexión	+200-500ms (primera vez), +50ms (reconexión)
Batería	Mínimo (cifrado por hardware en nRF52)